Sparse Cholesky Precision Approximation for Derivative-Enhanced Gaussian Processes: An Exploratory Summary

Sammie De Roberts

April 5, 2026

Abstract

This document summarizes an investigation into sparse Cholesky factorization of Gaussian process (GP) precision matrices, based on the method of Schäfer et al. (2021) and its extension to derivative-enhanced GPs by Logakannan et al. (2025). Through numerical experiments on the 1D Griewank function, we examine the full pipeline—MMD ordering, sparsity set construction via the paper's geometric fill-distance criterion, and column-wise submatrix solves—and assess how sparsity in the precision matrix approximation affects hyperparameter optimization and downstream predictions. We compare the paper's geometric sparsity criterion against a kernel-length-scale-based alternative, examine the interaction with input/output normalization, and find that derivative-enhanced GPs are remarkably robust to aggressive sparsification during hyperparameter optimization.

1 Background

The central computational bottleneck in GP regression is the $\mathcal{O}(N^3)$ cost of inverting the covariance matrix $\mathbf{K}$. When derivative observations are included up to order d in p dimensions, the augmented matrix $\mathbf{K}_{\text{der}}$ has dimension $N_d \times N_d$ where

$$N_d = \binom{p+d-1}{d} \cdot N, \tag{1}$$

making the $\mathcal{O}(N_d^3)$ cost quickly intractable. Sparse Cholesky factorization of the precision matrix $\mathbf{K}^{-1} \approx \mathbf{U}\mathbf{U}^\top$, where $\mathbf{U}$ is sparse upper triangular, reduces this cost to $\mathcal{O}(N \cdot m^3)$ where m is the average number of neighbors per column.

2 The Sparse Cholesky Pipeline

The algorithm proceeds in three stages, each feeding into the next.

2.1 Stage 1: MMD Ordering

The training points $\{x^{(i)}\}_{i=1}^N$ are reordered via maximum-minimum distance (MMD) ordering to produce a permutation $\mathbf{P}$ and fill-distances $\boldsymbol{\ell}$. At each step q, the algorithm selects the unordered point maximizing the minimum distance to all previously selected points:

$$P(q+1) = \arg \max_{i \notin \{P(1),\ldots,P(q)\}} \operatorname{dist}\left(x^{(i)}, \{x^{P(1)}, \ldots, x^{P(q)}\}\right). \tag{2}$$

The fill-distance for each point records the minimum distance to its predecessors at the time of insertion:

$$\ell^{(i)} = \text{dist}\left(x^{P(i)}, \{x^{P(1)}, \ldots, x^{P(i-1)}\}\right). \tag{3}$$

The resulting fill-distances decrease monotonically. This ordering ensures that well-separated points appear first (capturing large-scale structure) and closely-spaced points appear last (filling gaps), which produces near-sparsity in the Cholesky factor.

2.2 Stage 2: Sparsity Set Construction

For each column j of $\mathbf{U}$, the sparsity set $\mathcal{S}_j$ determines which predecessor rows $i < j$ receive nonzero entries. The paper uses a *geometric* criterion based on the fill-distances:

$$\mathcal{S}_j = \{(i, j) : i \leq j,\ \|x_{P(i)} - x_{P(j)}\| \leq \rho \cdot \ell^{(j)}\}, \tag{4}$$

where ρ is the sparsification parameter. A key practical advantage of this criterion is that the sparsity pattern depends only on the point geometry and ρ—it is *independent of the kernel hyperparameters*. This means the sparsity sets are computed once from the MMD ordering and reused across all NLML evaluations during hyperparameter optimization. Only the kernel submatrices $\mathbf{K}_{\text{sub}}$ need to be rebuilt at each candidate δ.

An alternative criterion thresholds against the kernel length scale, $\|x_{P(i)} - x_{P(j)}\| \leq \rho \cdot \delta$. While this is correlation-adaptive, it suffers from a bootstrapping problem: the sparsity pattern depends on δ, which is the quantity being optimized. The paper's geometric criterion avoids this entirely.

Interaction with normalization. The fill-distances $\ell^{(j)}$ scale with the point spacing. Under standard scaling ($\tilde{x} = (x - \mu_x)/\sigma_x$), the fill-distances shrink by a factor of σ_x. For a given ρ, the neighbor radius $\rho \cdot \ell^{(j)}$ shrinks proportionally. However, because the domain also shrinks by the same factor, the relative sparsity pattern is preserved: the same ρ value yields nearly identical sparsity percentages regardless of whether the data is normalized. This was confirmed in our experiments, where the sparsity at $\rho = 10$ was 19% for both the unnormalized and normalized versions of the large problem.

Supernode aggregation via λ. After the raw sparsity sets $\mathcal{S}_j$ are constructed, the paper aggregates columns with similar neighbor structures into *supernodes*, controlled by a parameter λ (set to 1.5 as suggested by Schäfer et al.). The full sparsity set is denoted $\mathcal{S}_{P,\ell,\rho,\lambda}$. Specifically, adjacent columns j and $j+1$ in the MMD ordering are grouped into the same supernode if their geometric locations satisfy

$$\|x_{P(j)} - x_{P(j+1)}\| \leq \lambda \cdot \ell^{(j+1)}. \tag{5}$$

Each supernode consists of a set of *parent* indices (the columns it owns) and *child* indices (the union of nonzero row entries across those columns). Crucially, supernode aggregation *expands* the effective sparsity pattern: each parent column is solved using the merged child set (the union of all individual neighbor sets within the supernode), which is generally larger than any individual column's neighbor set. This means that column-wise and supernode-based approaches produce *different* $\mathbf{U}$ factors—the supernode version has slightly more nonzero entries and can yield different (sometimes better) NLML landscapes.

The computational benefit is that columns within a supernode share the same $\mathbf{K}_{\text{sub}}$ factorization: the submatrix is extracted once for the supernode's combined child index set, factored once via LU decomposition at cost $\mathcal{O}(m^3)$, and then each parent column is solved via back-substitution

at cost $\mathcal{O}(m^2)$. This amortizes the dominant cubic cost across multiple columns. For our large test problem ($N = 20$), $\lambda = 1.5$ reduces 20 columns to 17 supernodes; for the derivative-enhanced system ($N_d = 40$), the same 17 supernodes replace 40 independent factorizations—a 57% reduction.

Supernode aggregation is structurally important in two further contexts:

- **Derivative-enhanced GPs:** Under point-wise ordering algorithm 1, each physical point's function value and derivative entries naturally form a supernode. The physical supernodes are expanded to derivative indices by mapping parent $i \to \{2i, 2i+1\}$ and similarly for children.
- **Dynamic updates (Section 4 of the paper):** The fixed/dynamic split for streaming data operates at the supernode level. When new data arrives, only the dynamic supernode's Cholesky factors are recomputed; all fixed supernodes are reused. Approach 1 merges all dynamic supernodes into one large supernode (less sparse but simpler updates), while Approach 2 maintains multiple dynamic supernodes and routes new points based on geometric proximity using the $\rho \cdot \ell^{(i)}$ radius.

2.3 Stage 3: Column-Wise and Supernode-Based Solves

The sparse factor $\mathbf{U}$ is found by minimizing the KL divergence $D_{\mathrm{KL}}\big(\mathcal{N}(0, \mathbf{K}) \,\|\, \mathcal{N}(0, (\mathbf{U}\mathbf{U}^\top)^{-1})\big)$, which decomposes column-by-column with a closed-form solution.

Column-wise approach. For column j with neighbor set $\mathcal{S}_j$:

1. Extract the small submatrix $\mathbf{K}_{\mathrm{sub}} = \mathbf{K}[\mathcal{S}_j, \mathcal{S}_j]$ from the original covariance matrix.
2. Solve the small linear system $\mathbf{K}_{\mathrm{sub}}\, \mathbf{v}_j = \mathbf{e}_j$, where $\mathbf{e}_j$ is the unit vector at the diagonal position of j within $\mathcal{S}_j$.
3. Normalize: $u_{jj} = \sqrt{v_j[\mathrm{diag}]}$, then $\mathbf{u}_j = \mathbf{v}_j / u_{jj}$.
4. Place $\mathbf{u}_j$ into the global $\mathbf{U}$ at the rows corresponding to $\mathcal{S}_j$.

Each column solve requires an independent LU factorization of $\mathbf{K}_{\mathrm{sub}}$ at cost $\mathcal{O}(|\mathcal{S}_j|^3)$, giving N factorizations and total cost $\mathcal{O}(N \cdot m^3)$. No full matrix inversion is ever performed.

Supernode approach. When supernodes are used, the procedure changes in two ways. First, the effective neighbor set for each parent column p in supernode k is the merged child set $\mathcal{C}_k$ (the union of all individual $\mathcal{S}_j$ across the supernode's parents), which is generally larger than any single $\mathcal{S}_p$. Second, the LU factorization of $\mathbf{K}[\mathcal{C}_k, \mathcal{C}_k]$ is computed *once* per supernode and reused for all parent columns via back-substitution:

1. Extract $\mathbf{K}_{\mathrm{sub}} = \mathbf{K}[\mathcal{C}_k, \mathcal{C}_k]$ and compute its LU factorization. $\mathcal{O}(|\mathcal{C}_k|^3)$
2. For each parent p in the supernode:
   (a) Solve $\mathbf{K}_{\mathrm{sub}}\, \mathbf{v}_p = \mathbf{e}_p$ using the cached LU factors. $\mathcal{O}(|\mathcal{C}_k|^2)$
   (b) Normalize: $u_{pp} = \sqrt{v_p[\mathrm{diag}]}$, $\mathbf{u}_p = \mathbf{v}_p / u_{pp}$.
   (c) Place $\mathbf{u}_p$ into $\mathbf{U}$ at the rows corresponding to $\mathcal{C}_k$.

The total cost is $\mathcal{O}\big(\sum_{k=1}^{n_{\mathrm{SN}}} |\mathcal{C}_k|^3 + N \cdot \bar{m}^2\big)$, where n_{SN} is the number of supernodes and $\bar{m}$ is the average child set size. This expression reveals an important tradeoff: supernodes reduce the *number* of factorizations from N to n_{SN}, but each factorization operates on a *larger* submatrix $|\mathcal{C}_k| \geq \max_{p \in \mathrm{SN}_k} |\mathcal{S}_p|$ because the child set is the union of all individual neighbor sets within the supernode. Since the factorization cost is cubic, this tradeoff is only favorable when the merged child set is not much larger than the individual sets—which occurs when adjacent columns share most of their neighbors. The λ parameter controls this: small λ only merges very close columns (similar neighbors, small expansion) while large λ merges distant columns (dissimilar neighbors, large expansion). In our experiments with $\lambda = 1.5$, the expansion was modest: for example, supernode 0 merged 4 parents with a child set of only 4 (no expansion), while the largest supernode had 2 parents with 15 children versus individual sets of $\sim$14 each.

Because the supernode uses a larger $\mathbf{K}_{\mathrm{sub}}$ (the merged child set) than the column-wise approach, the two methods produce *different* $\mathbf{U}$ factors—the supernode version has slightly more nonzero entries and can yield different NLML landscapes, as documented in Section 5.3.

Multivariate considerations. In multivariate settings with an ARD kernel having per-dimension length scales δ_k, the geometric criterion in Eq. (4) uses Euclidean distance in input space, which does not account for anisotropic correlation structure. This is by design: the criterion is purely geometric and does not require knowledge of the length scales, which are themselves the target of optimization. The theoretical guarantees from Schäfer et al. bound the approximation error in terms of ρ and the kernel's decay rate, so a sufficiently large ρ works regardless of the length scales—at the cost of reduced sparsity.

3 Role of Sparsity in the GP Workflow

A key finding from our experiments is that the sparse Cholesky approximation serves as an *optimizer accelerator*, not a model approximation. The workflow is:

1. **Hyperparameter optimization** (where sparse helps): Evaluate the negative log marginal likelihood (NLML) at many candidate δ values. Each evaluation requires $\mathbf{K}^{-1}$, which the sparse approximation provides cheaply. Critically, the sparsity pattern (from Eq. 4) is computed once and reused.

2. **Final training** (always exact): Once optimal δ^* is found, build the exact $\mathbf{K}$ and solve $\mathbf{K}\boldsymbol{\alpha} = \mathbf{f}$. Interpolation at training points is exact.

3. **Prediction** (same for both): $\mu(x^*) = \mathbf{k}(x^*, \mathbf{X})^\top \boldsymbol{\alpha}$.

The sparse approximation to the NLML is

$$\mathrm{NLML}_{\mathrm{sparse}} = \frac{1}{2}\|\mathbf{U}^\top \mathbf{f}\|^2 - \sum_j \log |U_{jj}| + \frac{N}{2}\log(2\pi), \tag{6}$$

where the first term approximates $\frac{1}{2}\mathbf{f}^\top \mathbf{K}^{-1}\mathbf{f}$ and the second approximates $\frac{1}{2}\log|\mathbf{K}|$. The question is whether $\mathrm{NLML}_{\mathrm{sparse}}(\delta)$ has the same minimum as $\mathrm{NLML}_{\mathrm{dense}}(\delta)$.

4 Derivative-Enhanced Covariance Structure

For the squared exponential kernel $k(x, y) = \exp\left(-\frac{|x-y|^2}{2\delta^2}\right)$ in 1D, the derivative-enhanced blocks are

$$K[f, f] = k(x_i, x_j), \tag{7}$$

$$K[f, \nabla f] = \frac{x_i - x_j}{\delta^2}\, k(x_i, x_j), \tag{8}$$

$$K[\nabla f, \nabla f] = \left(\frac{1}{\delta^2} - \frac{(x_i - x_j)^2}{\delta^4}\right) k(x_i, x_j). \tag{9}$$

Using the "point-wise ordering algorithm 1" from Logakannan et al., the derivative observations are interleaved with their parent point:

$$\mathbf{F} = [f(x_{P(1)}),\ \nabla f(x_{P(1)}),\ f(x_{P(2)}),\ \nabla f(x_{P(2)}),\ \ldots], \tag{10}$$

which keeps each point's function value and derivative together in the MMD ordering. This grouping preserves local derivative information under sparsification and enables incremental column updates for the digital twin streaming application.

When standard scaling is applied ($\tilde{x} = (x-\mu_x)/\sigma_x$, $\tilde{f} = (f-\mu_f)/\sigma_f$), the derivative observations must be scaled accordingly:

$$\widetilde{\nabla f} = \nabla f \cdot \frac{\sigma_x}{\sigma_f}. \tag{11}$$

5 Numerical Experiments

All experiments use the 1D Griewank function $f(x) = x^2/4000 - \cos(x) + 1$ with the paper's geometric sparsity criterion (Eq. 4). We test two configurations: a small problem ($[-\pi, \pi]$, $N = 6$) and a large problem ($[-10, 10]$, $N = 20$), both with standard scaling applied to inputs and outputs.

5.1 Forced Sparsity: Effect on Hyperparameter Optimization

Figure 1 shows the NLML landscapes and resulting predictions across a range of ρ values. Table 1 summarizes the optimal δ^* found by the sparse NLML at each sparsity level, with all final models trained using exact $\mathbf{K}$ at the respective δ^*.

5.2 Robustness of Derivative-Enhanced GPs to Sparsification

The most striking finding is the asymmetry in robustness between the standard GP and derivative-enhanced GP under aggressive sparsification. For the large problem ($N = 20$, $[-10, 10]$) with standard scaling:

- **No derivatives,** $\rho = 3$ (64% sparse): The sparse NLML shifts δ^* by -0.058, degrading MSE by $\sim 20\times$ (from 6.2×10^{-7} to 1.3×10^{-5}).
- **1st derivatives,** $\rho = 3$ (66% sparse): The sparse NLML shifts δ^* by only -0.039, and the MSE actually *improves* (6.9×10^{-13} vs. 1.8×10^{-12}).

This robustness arises because derivative observations provide strong local constraints that anchor the likelihood landscape. When distant point-pairs are dropped from $\mathbf{U}$ at low ρ, the no-derivative GP loses its only source of inter-point information (correlations), directly distorting the

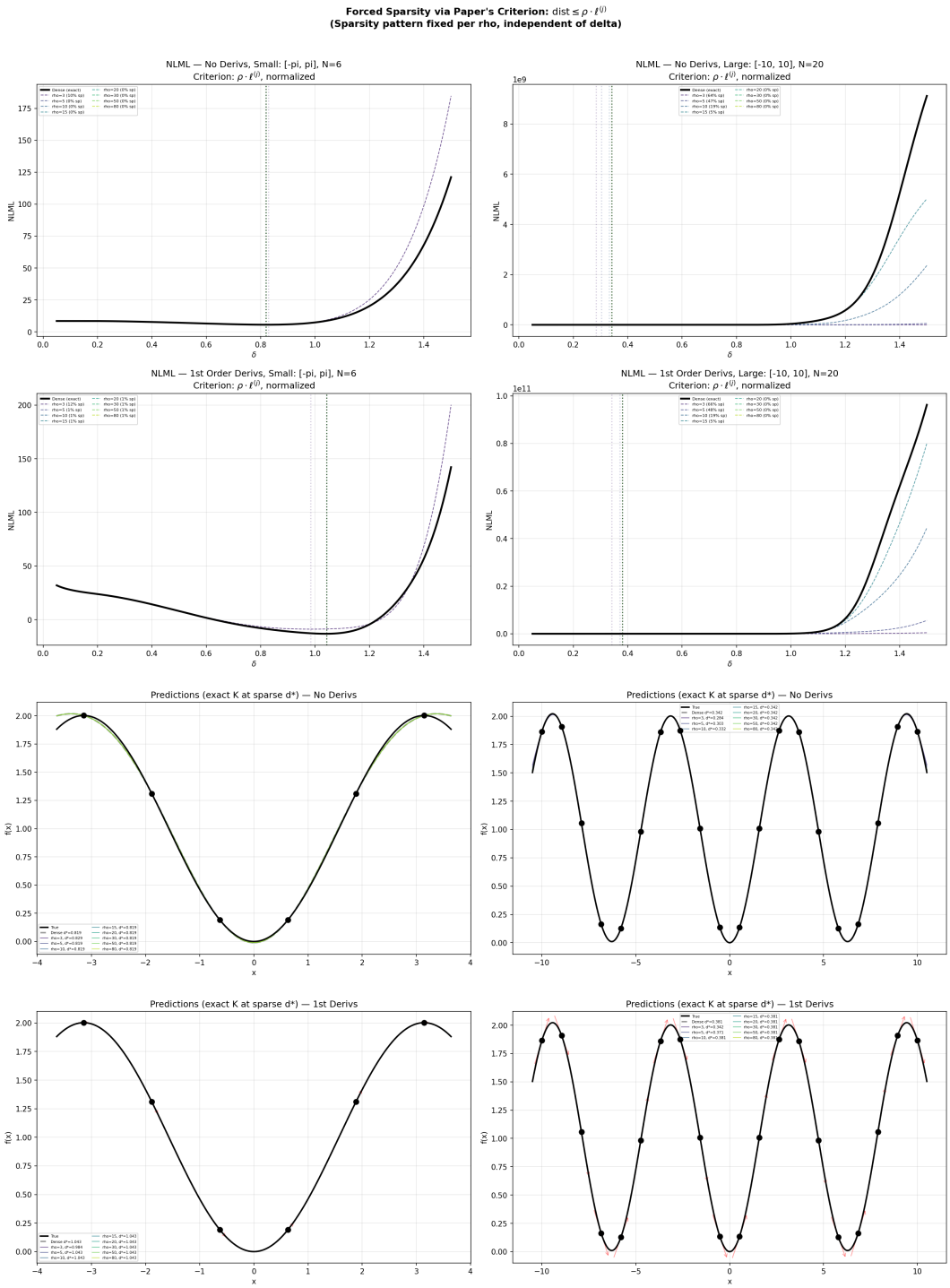

Figure 1: Effect of forced sparsity on NLML landscapes and predictions using the paper's geometric criterion (Eq. 4) with standard scaling. Left column: small problem ($[-\pi, \pi]$, $N = 6$). Right column: large problem ($[-10, 10]$, $N = 20$). Rows from top: NLML for no-derivative GP; NLML for derivative-enhanced GP; predictions from no-derivative GP at each ρ's optimal δ^*; predictions from derivative-enhanced GP. All predictions use exact $\mathbf{K}$ at the sparse-found δ^*.

Table 1: Optimal δ^* from sparse NLML optimization using the paper's geometric criterion with standard scaling. All final models are trained with exact $\mathbf{K}$.

	No Derivatives				1st Order Derivatives			
ρ	Sparsity	δ^*	Shift	Test MSE	Sparsity	δ^*	Shift	Test MSE
Small: $[-\pi, \pi]$, $N = 6$, *dense* $\delta^*_{nd} = 0.819$, $\delta^*_{1st} = 1.043$								
3	10%	0.829	+0.010	4.2e−4	12%	0.984	−0.058	8.8e−13
5	0%	0.819	+0.000	4.2e−4	1%	1.043	+0.000	7.4e−12
10	0%	0.819	+0.000	4.2e−4	1%	1.043	+0.000	7.4e−12
Large: $[-10, 10]$, $N = 20$, *dense* $\delta^*_{nd} = 0.342$, $\delta^*_{1st} = 0.381$								
3	64%	0.284	−0.058	1.3e−5	66%	0.342	−0.039	6.9e−13
5	47%	0.303	−0.039	5.8e−6	48%	0.371	−0.010	1.6e−12
10	19%	0.332	−0.010	1.2e−6	19%	0.381	+0.000	1.8e−12
15	5%	0.342	+0.000	6.2e−7	5%	0.381	+0.000	1.8e−12
20	0%	0.342	+0.000	6.2e−7	0%	0.381	+0.000	1.8e−12

NLML. The derivative-enhanced GP retains local curvature information at each point regardless of sparsification, so the NLML remains well-shaped.

By $\rho = 10$ (19% sparsity), the derivative-enhanced GP finds the exact dense δ^*, while the no-derivative GP still shows a small shift of -0.010. The paper's recommended $\rho = 10$–20 is well within the regime where both methods converge.

5.3 Column-Wise vs Supernode-Based Sparse Cholesky

We implemented the supernode aggregation described in Section 2.2 with $\lambda = 1.5$ and compared it against column-wise sparse Cholesky for the large problem ($N = 20$, $[-10, 10]$). Table 2 and Figure 2 summarize the results.

Table 2: Column-wise vs supernode comparison for the large problem ($[-10, 10]$, $N = 20$, $\lambda = 1.5$). Dense optima: $\delta^*_{\text{nd}} = 0.343$, $\delta^*_{\text{1st}} = 0.372$. Supernodes reduce factorization count from N (or $2N$) to 17 across all ρ values. "SN" = supernode.

	No Derivatives				1st Order Derivatives			
ρ	δ^*_{col}	δ^*_{SN}	MSE_{col}	MSE_{SN}	δ^*_{col}	δ^*_{SN}	MSE_{col}	MSE_{SN}
3	0.284	0.284	1.2e−5	1.2e−5	0.343	0.343	7.7e−13	7.7e−13
5	0.299	0.299	6.7e−6	6.7e−6	0.372	0.358	1.8e−12	1.1e−12
10	0.328	0.328	1.5e−6	1.5e−6	0.372	0.358	1.8e−12	1.1e−12
15	0.343	0.343	5.6e−7	5.6e−7	0.387	0.358	2.0e−12	1.1e−12
20	0.343	0.343	5.6e−7	5.6e−7	0.372	0.358	1.8e−12	1.1e−12

Three observations emerge from this comparison. First, for the no-derivative GP, column-wise and supernode approaches find *identical* δ^* and MSE at every ρ. The expanded neighbor sets from supernode aggregation add entries that are too weakly correlated to shift the NLML minimum. Second, for the derivative-enhanced GP, the supernode approach consistently finds $\delta^* = 0.358$ across $\rho = 5$–20, compared to the column-wise $\delta^* = 0.372$. The supernode's expanded child sets give the derivative system access to additional cross-point derivative correlations, subtly reshaping the NLML landscape and yielding a slightly better MSE (1.1×10^{-12} vs 1.8×10^{-12}). Third, the

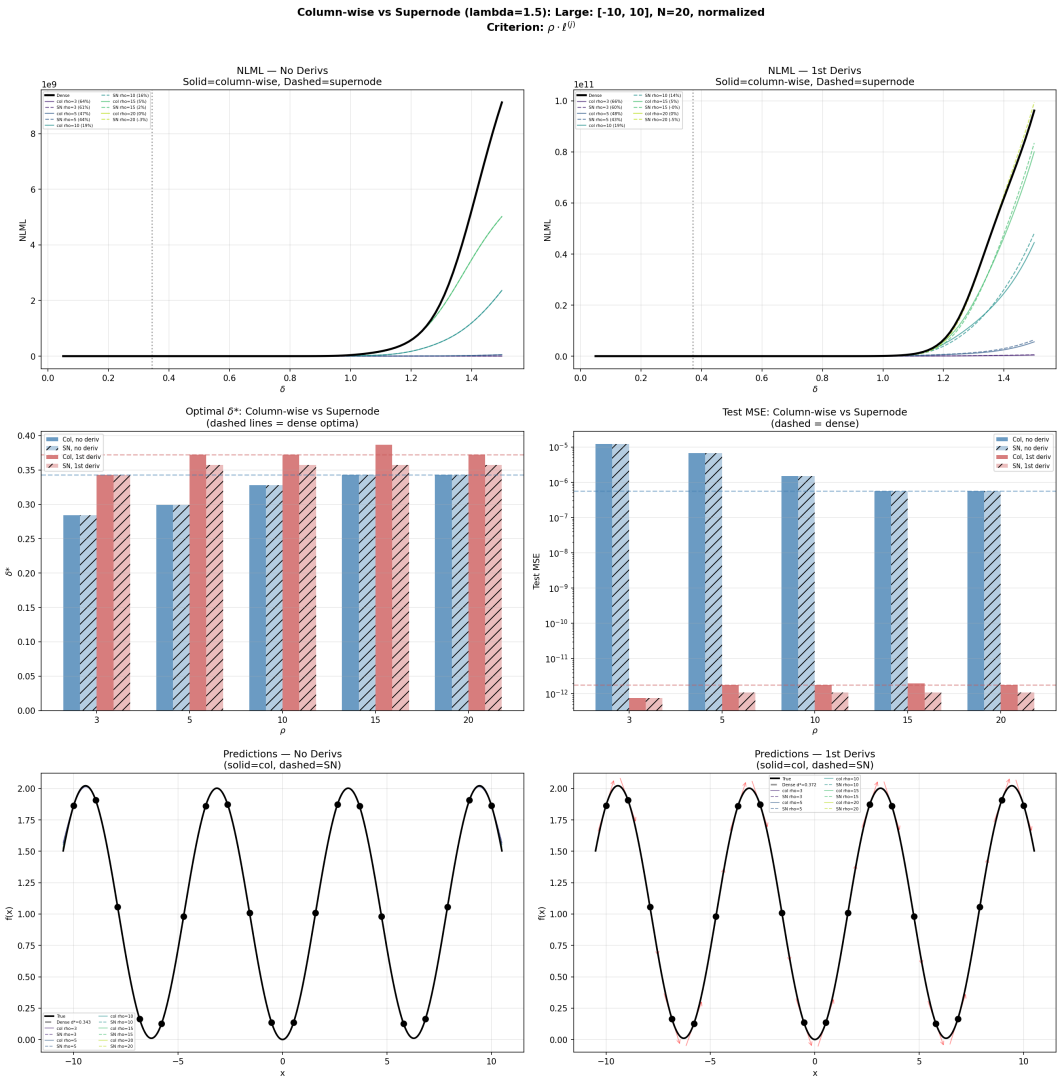

Figure 2: Column-wise vs supernode comparison for the large problem ($[-10, 10]$, $N = 20$, $\lambda = 1.5$). Top row: NLML landscapes with column-wise (solid) and supernode (dashed) curves overlaid. Middle row: optimal δ^* and test MSE across ρ values. Bottom row: resulting predictions in the original (unscaled) domain. All predictions use exact $\mathbf{K}$ at the respective δ^*.

factorization count is fixed at 17 supernodes regardless of ρ, compared to 20 factorizations for column-wise without derivatives and 40 for column-wise with derivatives—a 57% reduction in the derivative case.

5.4 Effect of Standard Scaling

Standard scaling ($\tilde{x} = (x - \mu_x)/\sigma_x$, $\tilde{f} = (f - \mu_f)/\sigma_f$) has three important effects:

1. **Brings δ^* into a manageable range.** Without scaling, $\delta^* \approx 2.4$ for the large domain; with scaling, $\delta^* \approx 0.34$–0.38.

2. **Improves conditioning.** For the small problem, $\kappa(\mathbf{K})$ drops from $\sim 10^{10}$ (unscaled) to $\sim 10^2$ (scaled, no derivatives). The derivative-enhanced system remains poorly conditioned ($\sim 10^9$) due to the inherent δ^{-4} scaling of $K[\nabla f, \nabla f]$ blocks, consistent with Lemma 3.0.2 of Logakannan et al.

3. **Preserves sparsity structure.** Since the MMD fill-distances and the domain both scale by σ_x, the geometric sparsity criterion produces nearly identical sparsity percentages at the same ρ regardless of normalization.

6 Connection to JetGP

The sparse Cholesky framework is complementary to JetGP in several respects. JetGP handles arbitrary-order derivative kernel computation via OTI algebra, while the sparse Cholesky addresses the $\mathcal{O}(N_d^3)$ scaling bottleneck. A natural integration would use JetGP to compute the $\mathbf{K}_{\text{sub}}$ entries needed for each supernode solve, enabling scalable derivative-enhanced GPs with automatic kernel differentiation rather than manual derivation of kernel derivative blocks. Specifically, at each candidate δ during hyperparameter optimization, JetGP would evaluate the SE kernel and all required derivative blocks at the indices specified by each supernode's child set, and the sparse Cholesky machinery would factorize and solve to produce the approximate NLML.

The supernode structure is particularly well-matched to JetGP's point-wise derivative computation. Under point-wise ordering algorithm 1, each physical point's function value and derivative entries are grouped into the same supernode. JetGP already computes kernel derivatives on a per-point basis via OTI algebra, so the supernode's $\mathbf{K}_{\text{sub}}$ can be assembled directly from JetGP's per-point kernel evaluations without redundant computation. For a GP with derivatives up to order d in p dimensions, each physical point contributes $\binom{p+d-1}{d}$ entries to the supernode, and the supernode's child set defines exactly which cross-point kernel blocks are needed. The factorization savings from supernodes compound with JetGP's automatic differentiation: rather than manually deriving and implementing each $K[\nabla^n f, \nabla^m f]$ block (as done in Logakannan et al. for the SE kernel up to 4th order), JetGP would compute them automatically for any kernel and any derivative order.

The robustness finding from our experiments further strengthens this integration. Since derivative-enhanced GPs tolerate aggressive sparsification during hyperparameter optimization, the combined approach could use moderate ρ values (e.g., $\rho = 5$–10, yielding 20–50% sparsity) for fast NLML evaluation. The paper's geometric criterion (Eq. 4) is especially well-suited because the sparsity pattern—and therefore the supernode structure—is computed once from the MMD ordering and reused across all NLML evaluations, with no dependence on the kernel hyperparameters being optimized. Only the kernel submatrix entries within each supernode need to be recomputed at each candidate δ, which is exactly what JetGP is designed to do efficiently.

7 Conclusions

1. The sparse Cholesky precision approximation functions as an optimizer accelerator for GP hyperparameter search, not as a final model approximation. The final GP is always trained with exact $\mathbf{K}$, ensuring exact interpolation at training points.

2. The paper's geometric sparsity criterion ($\rho \cdot \ell^{(j)}$) has a critical practical advantage: the sparsity pattern is independent of the kernel hyperparameters and can be computed once from the MMD ordering. This avoids the bootstrapping problem inherent in kernel-length-scale-based criteria, and extends naturally to multivariate ARD kernels without requiring knowledge of per-dimension length scales.

3. Derivative-enhanced GPs are significantly more robust to sparsification than standard GPs. At 66% sparsity ($\rho = 3$), the derivative-enhanced NLML landscape shifts δ^* by only 0.039 with no degradation in MSE, while the standard GP at equivalent sparsity degrades MSE by $20\times$.

4. Supernode aggregation ($\lambda = 1.5$) expands the effective sparsity pattern by merging neighbor sets across grouped columns. This does not change the result for standard GPs but subtly improves the derivative-enhanced GP, which consistently finds a slightly better δ^* with the expanded neighbor sets. The primary benefit is computational: supernodes reduce the number of LU factorizations by 57% for the 1D derivative-enhanced case, with greater savings expected at higher derivative orders and dimensions.

5. Standard scaling preserves the geometric sparsity structure (identical sparsity percentages at the same ρ) while improving conditioning and bringing δ^* into a manageable range. Derivative observations must be scaled by σ_x/σ_f.

6. For the 1D Griewank function, first-order derivatives improve MSE by factors of 10^5–10^7 across all tested configurations, consistent with the results reported in Logakannan et al.

References

1. F. Schäfer, M. Katzfuss, and H. Owhadi, "Sparse Cholesky Factorization by Kullback–Leibler Minimization," *SIAM J. Sci. Comput.*, vol. 43, no. 3, pp. A2019–A2046, 2021.

2. K. P. Logakannan, S. Vashishtha, J. Hochhalter, S. Zhe, and R. M. Kirby, "A Streaming Sparse Cholesky Method for Derivative-Informed Gaussian Process Surrogates Within Digital Twin Applications," *arXiv:2511.00366v1*, 2025.

3. Y. Chen, B. Hosseini, H. Owhadi, and A. M. Stuart, "Solving and Learning Nonlinear PDEs with Gaussian Processes," *arXiv:2103.12959*, 2021.